

FEM Homework

Jules Herrmann

29. March 2026

In this report, I explain my work for the homework of the FEM class.
I chose to work on Project 4 : Elimination tree and direct sparse Cholesky solver.
I also implemented another sphere mesh, based on the geodesic subdivision of an icosahedron.

1 Sparse Cholesky Decomposition

This work follows the method proposed in J. Scott and M. Tuma [1].

Let's denote by A^a the “anti-transpose” of the matrix A , which is the matrix obtained by mirroring along the anti-diagonal. More formally $A^a := \tilde{I}A^T\tilde{I}$ where $\tilde{I}$ is a matrix with only ones on the antidiagonal. Any linear problem involving A^T can be solved using A^a , as shown in the following computation :

$$A^T x = b \Leftrightarrow \tilde{I}A^T\tilde{I}\tilde{I}x = \tilde{I}b \Leftrightarrow A^a x^r = b^r \quad (1.1)$$

where $v^r := \tilde{I}v$ is the vector v in reversed order.

This is useful because, if L is lower sub-diagonal, then L^T is upper subdiagonal, but L^a is lower subdiagonal.

This allow to solve linear problem involving LL^T using only functions that work for lower subdiagonal matrices, saving me a bit of implementation work.

All the sparse matrices are stored using the Compressed Sparse Row (CSR) scheme. This data-structure doesn't allow for fast iterations over a column, but we can use it to iterate over a row efficiently.

1.1 Symbolic phase

Function `elimination_tree` is my implementation of algorithm 4.2 of [1].

The pseudo code uses an indexing scheme starting at 1 and uses 0 to denote the absence of a parent. Since I transcribed the algorithm to use 0-based indexing, I chose to denote the absence of parent with the maximum integer `UINT32_MAX`.

Function `cholesky_sparsity_pattern` is my implementation of algorithm 4.3 of [1].

If A is the input sparse matrix, and LL^T its Cholesky factorization, then this function returns both the sparsity pattern of L and L^a . It proceeds by creating the graph of L as an adjacency list (resp. `rowL` and `colL`). The graphs are filled in according the algorithm, then the sparsity patterns is built from the graphs.

1.2 Sparse triangular solver

Let's first describe the data structure used to store a sparse vector.

Let v be a vector with nnz non-zero coefficients. It is described by :

- A buffer v , of type `double*`, containing contiguously all the non-zero coefficients
- A buffer v_ind , of type `uint32_t*`, contains the index of the non-zero coefficients
- An integer v_size stores nnz .

This data scheme must satisfy that if $v_i = \alpha$, then there exist a k such that $v[k] = \alpha$ and $v_ind[k] = i$.

Two functions `sparse_tri_solve` have been implemented :

- One deals with dense vectors and a sparse matrix
- One deals with both sparse vectors and a sparse matrix

Both, given a lower subdiagonal matrix L and a vector b , find a vector x such that $Lx = b$.

The algorithm is a simple forward substitution algorithm, where, at step row , we compute

$$x_{\text{row}} = \frac{b_{\text{row}} - \sum_{\text{col}=1}^{\text{row}-1} L_{\text{row},\text{col}} x_{\text{col}}}{L_{\text{row},\text{row}}} \quad (1.2)$$

Notice that, in this algorithm, we only iterate over consecutive values of L row-wise, which can be done efficiently on CSR matrices.

The only difficulty is the computation of the sum in the case of sparse vectors. We want to iterate over x and a row of L , which are both sparse vectors, so some care is required for the iteration to stay synchronized.

1.3 Numerical phase

The numerical phase of the Cholesky factorization uses the *up-looking* cholesky algorithm.

Indeed this algorithm can be implemented in such a way that the only iterations over consecutive values of the matrix happen along a row, which can be done efficiently on CSR matrices.

Up-looking Cholesky factorization

```

1  $L_{11} \leftarrow (A_{11})^{\frac{1}{2}}$ 
2 for  $i$  from 2 to  $N$ 
3   | Solve  $L_{1:i-1,1:i-1} z = A_{i,1:i-1}$ 
4   |  $L_{i,1:i-1} \leftarrow z$ 
5   |  $L_{i,i} \leftarrow (A_{ii} - zz^T)^{\frac{1}{2}}$ 
6 end
```

In line 3, the triangular solver that was presented in Section 1.2 is used.

Since the rows of A , as well as their indices, are stored contiguously in memory, no data movement is required to call the sparse triangular solver at line 3.

The size of the sparse vector, that is, the number of non-zero coefficients of index inferior to i , still needs to be computed, which I did with a linear search, but could be done with a dichotomic search since the indices are sorted.

Function `sparse_cholesky` is my implementation of the algorithm.

1.4 Sparse Cholesky solver

I created a `CholeskySolver` struct that contains the lower subdiagonal matrix L , as well as L^a .

As explained earlier, L^a can be used to solve linear problem with L^T , but this doesn't require to implement backward substitution. Therefore the following algorithm is used to solve $Ax = b$

Cholesky solver

- 1 Solve $Ly = b$
 - 2 Solve $Lz = y^r$
 - 3 return z^r
-

The method `CholeskySolver::solve` implements this algorithm.

A method `CholeskySolver::update_same_pattern` has also been implemented which allows to update the coefficients of the solver without recomputing the sparsity patterns. This is required because the interface of the demonstration program allows the user to change the value of dt and ν .

2 Better Sphere mesh

The proposed solution of generating a sphere mesh by projecting the points of a subdivided cube on the sphere yields a very anisotropic mesh with wide variations of the area of faces.

I implemented an algorithm to produce the mesh of a geodesic subdivided sphere. The idea is to start from a regular triangular mesh, such as an icosahedron, and recursively subdivide each triangular face according to the following diagram :

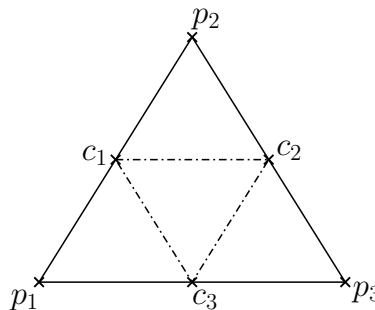


Figure 1: Subdivision scheme for a triangular face

At each iteration, each newly created vertex is projected on the sphere.

To avoid duplicated vertex, before adding a child vertex c between two parent vertices p_1 and p_2 , we need to check if a vertex had been previously created between those parent vertices, and retrieve its index if needed.

This requires the use of a datastructure mapping sets of two parent vertices $\{p_1, p_2\}$ and the index of their child vertex.

I used a buffer of size N^2 , with N the number of vertex before subdivision. A couple $\{p_1, p_2\}$ was assigned the index $Np_1 + p_2$ if $p_1 < p_2$ and $Np_2 + p_1$ otherwise.

This is a very inefficient use of computer memory, but this scheme is very easy to implement and good enough for this purpose since the data structure is only used shortly at the start of the program.

Bibliography

- [1] J. Scott and M. Tũma, *Algorithms for Sparse Linear Systems*. in Nečas Center Series. Springer International Publishing, 2023.